

Pemrograman Berorientasi Objek

Dany Candra Febrianto, S.Kom., M.Eng.

Minggu 05 : Polimorphysm, Casting dan Instance Of

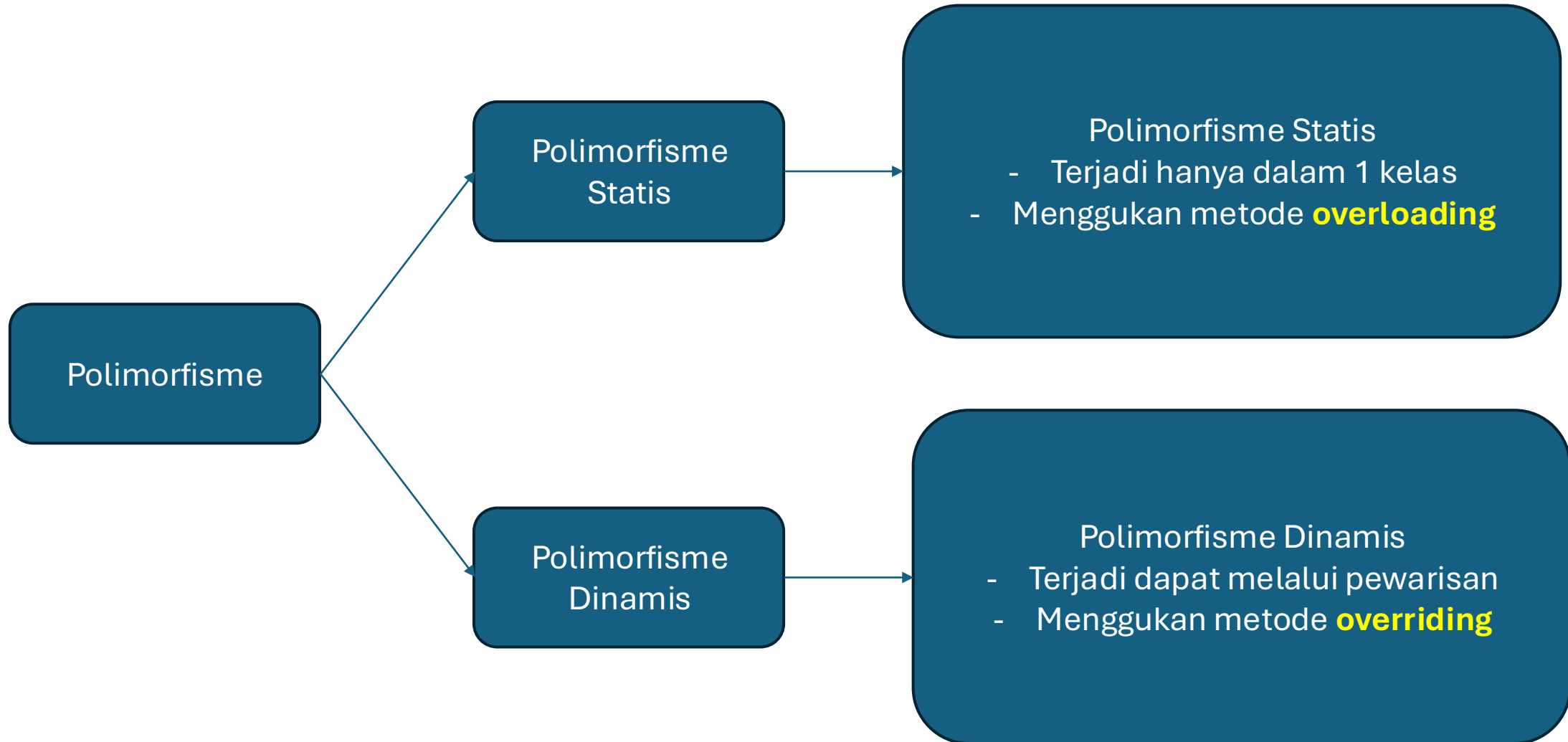
Agenda

- Polymorphism
- Casting & Instance Of

Polimorfisme

- Polimorfisme (bahasa inggris polymorphism) adalah sebuah prinsip dalam biologi di mana organisme atau spesies dapat memiliki banyak bentuk atau tahapan (stages).
- Polimorfisme dalam OOP adalah sebuah prinsip di mana class dapat memiliki banyak “bentuk” prosedur/fungsi yang berbeda-beda meskipun namanya sama.
- **“Bentuk” di sini dapat diartikan: isinya berbeda, parameternya berbeda, dan tipe datanya berbeda.**

Jenis Polimorfisme



Contoh Polymorphism Statis

```
public class Matematika {  
  
    int tambah(int nilai1, int nilai2){  
        System.out.println("tambah() 1");  
        int hasil = nilai1 + nilai2;  
        return hasil;  
    }  
  
    int tambah(int nilai1, int nilai2, int nilai3)  
{  
        System.out.println("tambah() 2");  
        int hasil = nilai1 + nilai2 + nilai3;  
        return hasil;  
    }  
  
    double tambah(double nilai1, double nilai2){  
        System.out.println("tambah() 3");  
        double hasil = nilai1 + nilai2;  
        return hasil;  
    }  
  
}
```

Contoh Polymorphism Dinamis

```
public class Orang {  
  
    String nama;  
  
    Integer umur;  
  
    String alamat;  
  
    void sayHallo() {  
        System.out.println("Nama saya " + nama +  
            " alamat saya " + alamat +  
            " umur saya " + umur);  
    }  
}
```

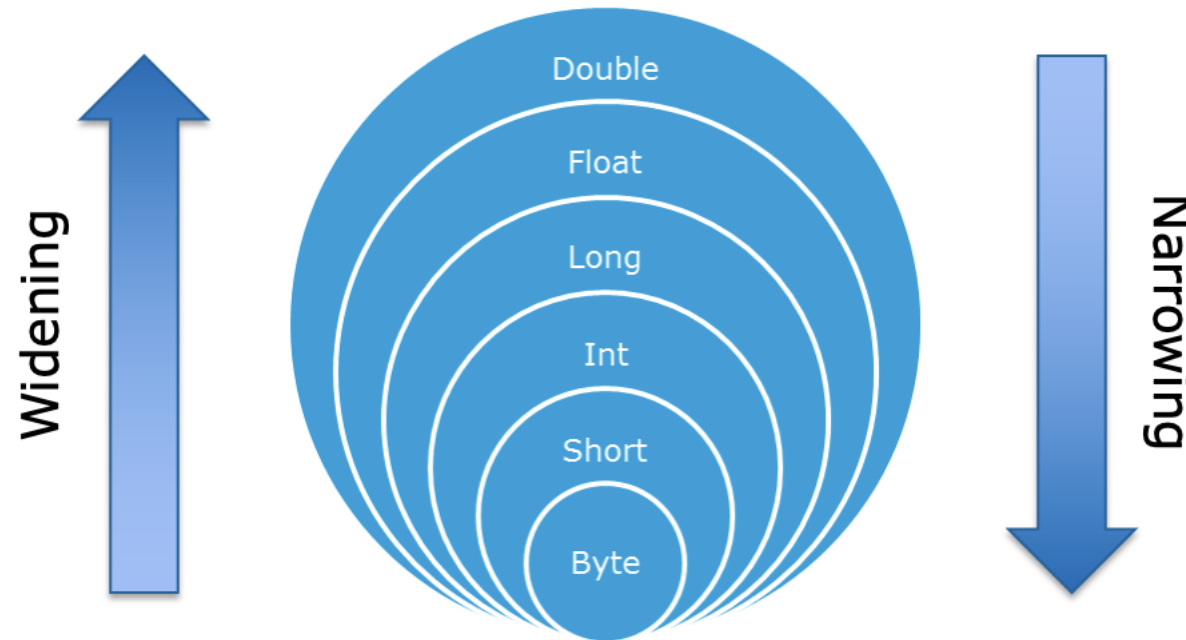
```
public class Mahasiswa extends Orang{  
  
    String nim;  
  
    @Override  
    void sayHallo() {  
        super.sayHallo();  
        System.out.println("Ini prosedur sayHallo(): "  
            + "yang berasal dari kelas mahasiswa");  
    }  
}
```

Kaitan Polymorfisme dan Pewarisan

- Polimorfisme sangat erat dengan pewarisan
- Dalam hal ini polymorfisme jg dapat disebut kemampuan sebuah kelas bertingkah laku atau memiliki sifat seperti kelas turunannya.
- Dengan menggunakan polimorfisme, sebuah kelas dapat disamarkan menjadi kelas parentnya.

Casting dan Instance Of

- Casting merupakan memasukkan suatu nilai dari salah satu tipe terhadap suatu variable dari tipe lain.



Widening :

bersifat implisit, konversi tipe secara otomatis kedua tipe tersebut harus kompatibel dan tipe dari target harus lebih besar dari pada sumber.

```
public class Driver{  
    public static void main(String args[]){  
        int i = 50;  
        long l = i;  
        float f = l;  
  
        System.out.println("int value: " + i);  
        System.out.println("long value: " + l);  
        System.out.println("float value: " + f);  
    }  
}
```

```
> int value: 50  
> long value: 50  
> float value: 50
```

byte → short → int → long → float → double



Widening

Narrowing :

Kebalikan dari widening, tipe dan target harus lebih kecil dari tipe data sumber.

```
public class Driver{  
    public static void main(String args[]){  
        double d = 25.16;  
        long l = (long)d;  
        int i = (int)l;  
  
        System.out.println("double value: " + d);  
        System.out.println("long value: " + l);  
        System.out.println("int value: " + i);  
    }  
}
```

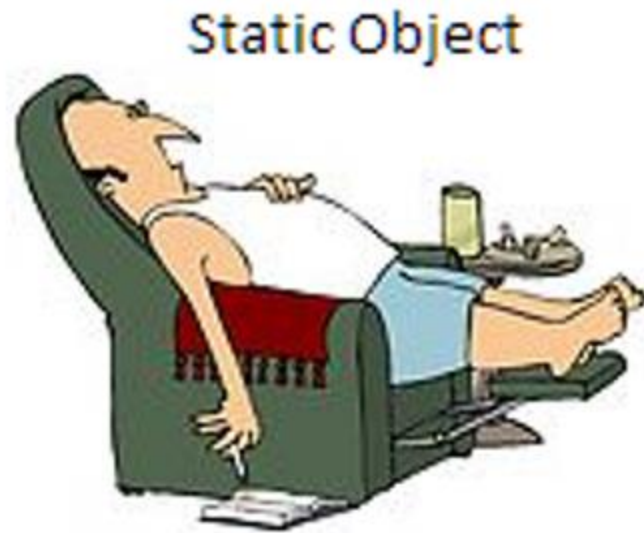
```
> double value: 25.16  
> long value: 25  
> int value: 25
```

byte ← short ← int ← long ← float ← double



Narrowing

Keyword Static



Static modifier

- Static Variable
- Static Method

Dengan menggunakan kata kunci static, kita bisa membuat field, method atau class bisa diakses langsung tanpa melalui object nya

Contoh Static

```
public class Application { 1 usage
```

```
    public final static String APP_NAME = "Belajar Java Application";
```

```
    public final static int PROCESSORS; 2 usages
```

```
    static {
```

```
        PROCESSORS = Runtime.getRuntime().availableProcessors();
```

```
    }
```

```
    public static int sum(int... numbers) { no usages
```

```
        int sum = 0;
```

```
        for (int number : numbers) {
```

```
            sum += number;
```

```
        }
```

```
        return sum;
```

```
    }
```

Static Variable

Static Block

Static Method

```
}
```

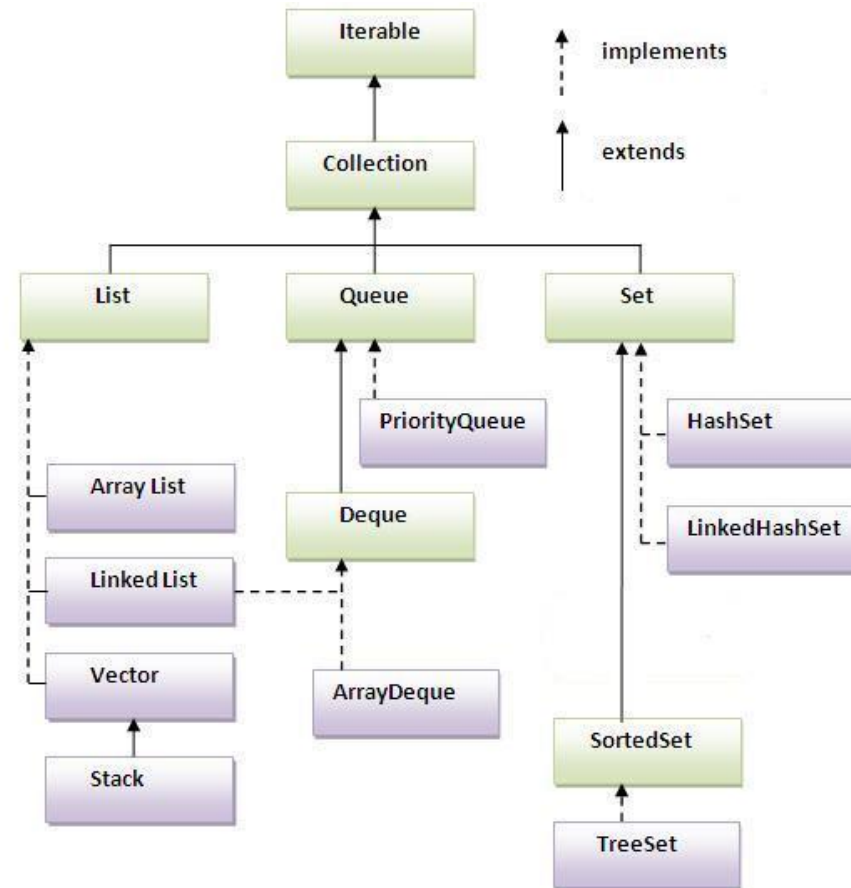
Penggunaan

```
public class AppMain {  
  
    public static void main(String[] args) {  
  
        // memanggil variabel static  
        System.out.println(Application.APP_NAME);  
  
        // memanggil method static  
        System.out.println(Application.sum(...numbers: 1,2,3,4,5,6));  
  
        // memanggil static block  
        System.out.println(Application.PROCESSORS);  
  
    }  
}
```

Java Collection

- Collection / Container adalah object yang mengumpulkan data-data (mirip array). Collection di java adalah generic class sehingga kita bisa mengisi data apapun dalam collection.

Java Collection Interface



Fungsi default

- add(item)
- remove(item)
- contains(item)

Untuk menampilkan Data For Loop

```
// Data
List<String> mahasiswa = new ArrayList<>(Arrays.asList(
    "Andi", "Budi", "Citra", "Dewi", "Eko"
));

// =====
// 1. FOR LOOP BIASA (index-based)
// =====
printHeader( judul: "1. FOR LOOP BIASA");
for (int i = 0; i < mahasiswa.size(); i++) {
    System.out.println("Index " + i + " : " + mahasiswa.get(i));
}
```

Untuk menampilkan Data For Each

```
// =====  
// 2. FOR EACH / ENHANCED FOR LOOP  
// =====  
printHeader( judul: "2. FOR EACH (Enhanced For Loop)");  
for (String nama : mahasiswa) {  
    System.out.println("Nama: " + nama);  
}
```

Untuk menampilkan Data Iterator

```
// =====  
// 3. ITERATOR  
// =====  
printHeader( judul: "3. ITERATOR");  
Iterator<String> iterator = mahasiswa.iterator();  
while (iterator.hasNext()) {  
    String nama = iterator.next();  
    System.out.println("Nama: " + nama);  
}
```

Untuk menampilkan Data

Lambda Expression

```
// =====  
// 4. LAMBDA (forEach dengan lambda expression)  
// =====  
printHeader(judul: "4. LAMBDA");  
mahasiswa.forEach(String nama -> System.out.println("Nama: " + nama));  
  
// Lambda dengan kondisi (filter pakai stream)  
printHeader(judul: "4b. LAMBDA + STREAM (filter nama > 4 huruf)");  
mahasiswa.stream()  
    .filter(String nama -> nama.length() > 4)  
    .forEach(String nama -> System.out.println("Nama panjang: " + nama));
```

Untuk menampilkan Data Reference

```
// =====  
// 5. METHOD REFERENCE  
// =====  
printHeader( judul: "5. METHOD REFERENCE");  
mahasiswa.forEach(System.out::println);  
  
// Method reference dengan stream dan sort  
printHeader( judul: "5b. METHOD REFERENCE + SORT");  
mahasiswa.stream()  
    .sorted(String::compareTo)  
    .forEach(System.out::println);
```

Terima Kasih